



AGRI-CREDIT RISK INTELLIGENCE

GramOS

Master Blueprint & Engineering Manual

Version 2.0 Real Buyer-Payment Risk Pivot

*Satellite imagery sees a healthy, green crop.
It cannot see that the sugar mill 30 km away
has not paid the farmer for four months.*

FOUNDER & MAINTAINER

Sudarshan Kulkarni

PRIMARY REGION

Karnataka — Belagavi, Bagalkot, Vijayapura

DOCUMENT DATE

31 July 2026

CLASSIFICATION

Internal — Founding Technical Reference

Detecting household default risk 60–120 days before the missed EMI.

Data provenance: Karnataka Cane Commissionerate public records · Deterministic scoring, RBI Model Risk Management aligned

Contents

Contents

The Core Thesis

1.1 The Problem

1.2 The GramOS Advantage

1.3 Signal Comparison

Technical Stack & Architecture

2.1 Stack Summary

2.2 System Architecture

Foundation & Core Engine

Phase 1 — India Stack & Agri-FinTech Terminology Refactor

Phase 2 — RBI-Compliant Deterministic Engine & Parallel LangGraph

Phase 3 — Supabase PostgreSQL Integration

Phase 4 — Time-Series Data Generator (30-Day Historical Backfill)

Phase 5 — Rewire Backend to Supabase & Dual-Axis Timeline UI

Security, Intelligence & Autonomy

Phase 6 — Supabase Auth & JWKS Public Key JWT Verification

Phase 7 — Multimodal Document Intelligence (Gemini 2.5 Flash Vision)

Phase 8 — Human-in-the-Loop Audit Log & Compliance Overrides

Phase 9 — Proactive Background Risk Alerting

Sugarcane Arrears & Buyer Risk Engine

Phase 10 — Cane Arrears & Catchment Database Integration

Seed Data — Karnataka 2025–26 Figures

Phase 11 — Integrating buyer_payment_risk into Scoring & LangGraph

Phase 12 — Buyer Payment Risk UI & Live Production Deployment

Scoring Methodology

6.1 Weighted Arrears Exposure

6.2 Buyer Risk Penalty

6.3 Final Deterministic Risk Score	16
6.4 Component Summary	16
Production Readiness Checklist	17
Schema & Endpoint Index	18
A.1 Database Tables	18
A.2 API Endpoints	18
A.3 Environment Variables	18

01 EXECUTIVE SUMMARY

The Core Thesis

GramOS is an agricultural credit-risk intelligence platform built for lenders operating in buyer-dependent farm supply chains. It introduces a credit signal that satellite-first platforms structurally cannot observe: Buyer Payment Risk.

1.1 The Problem

Traditional agricultural lending intelligence platforms (for example SatSure and Cropin) rely almost exclusively on satellite Earth observation: crop health, acreage, soil moisture, and plot-level NDVI.

This creates a massive structural blind spot. **Satellite imagery sees healthy, green crops. It cannot see that the sugar mill 30 km away has not paid the farmer for four months.**

In sugarcane and other buyer-dependent supply chains, growers are paid by processors and buyers, not directly by open markets. Under law, sugar mills must pay within 14 days of supply. In practice, payment delays stretch for months. When a mill defaults or delays payments, the impact lands across its entire catchment — thousands of farming households experience a severe, synchronised cash-flow shock at the same moment.

Why this matters to a lender: conventional models treat each borrower as an independent risk. A mill arrears event is a *correlated* shock. An entire taluk-level book can deteriorate simultaneously, which is precisely the failure mode that portfolio provisioning is least prepared for.

1.2 The GramOS Advantage

GramOS introduces a new credit risk signal: **Buyer Payment Risk**. By mapping sugar mill arrears — publicly reported by the Cane Commissionerate — to their catchment taluks and borrower pincodes, GramOS detects household default risk **60 to 120 days before** it manifests as a missed EMI on a lender's balance sheet.

```
[ Cane Commissioner / Public Data ] → [ Mill Arrears Position ]
                                     |
[ Borrower Credit Risk Score ] ← [ Catchment Taluk Exposure ]
(Lead time: 60–120 days before loan default)
```

Signal flow: public arrears disclosure resolves to a mill position, which resolves to catchment exposure, which prices into the borrower score.

1.3 Signal Comparison

Dimension	Satellite-first platforms	GramOS
Primary signal	NDVI, soil moisture, acreage, crop health	Mill arrears position mapped to catchment taluks, alongside climate and financial signals
Blind spot	Cannot observe buyer-side payment failure	Explicitly prices buyer-side payment failure into the score

Dimension	Satellite-first platforms	GramOS
Risk shape	Treats borrowers as independent	Captures correlated, catchment-wide cash-flow shocks
Lead time	Detected at or near the missed EMI	60–120 days ahead of the missed EMI
Provenance	Commercial EO imagery	Karnataka Cane Commissionerate public records

02 PLATFORM

Technical Stack & Architecture

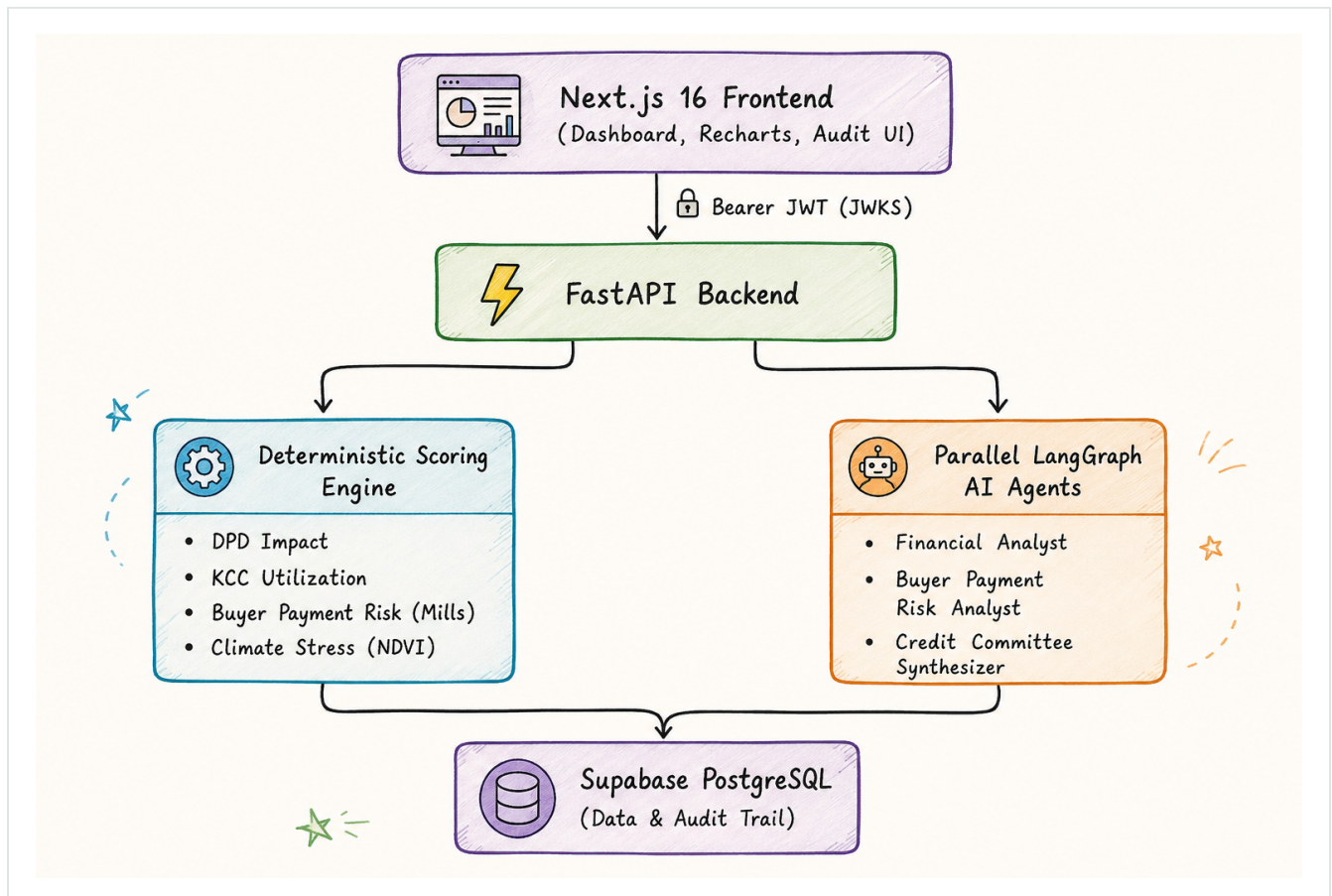
2.1 Stack Summary

Layer	Technology
Frontend	Next.js 16 (App Router, React 19), Tailwind CSS, Recharts (dual-axis time series), Lucide React
Backend	FastAPI (Python 3.11+), Uvicorn, Pydantic v2, PyJWT (JWKS public key validation), APScheduler (background worker)
AI & multi-agent engine	LangGraph (StateGraph parallel fan-out / fan-in), Google Gemini 2.5 Flash (google-genai SDK, native multimodal vision)
Database	Supabase PostgreSQL — enterprises, financial_ledgers, climate_snapshots, mills, catchments, document_insights, audit_logs, alerts
Auth	Supabase Auth, cookie-backed @supabase/ssr
Scoring methodology	100% deterministic mathematical score plus LLM-generated Explainable AI (XAI) narrative — ensuring full compliance with RBI Model Risk Management principles

The non-negotiable architectural rule. The number is computed by mathematics; the language is generated by the model. Gemini never authors the risk score — it explains one. The `synthesize_risk` node overwrites the model output with the deterministic score as a hard backstop against hallucination.

2.2 System Architecture

Authenticated requests fan out from the FastAPI backend into two independent paths — the deterministic scoring engine and the parallel LangGraph agent set — which converge on Supabase for persistence and the audit trail.



GramOS end-to-end system design.

03 BUILD SEQUENCE · PART I

Foundation & Core Engine

The complete sequence of prompts below reconstructs or expands the GramOS platform step by step. Each phase is self-contained and assumes the previous phase has been verified. Run them in order.

Phase 1 — India Stack & Agri-FinTech Terminology Refactor

Objective. Align GramOS terminology and data models with the Indian Agri-FinTech ecosystem — KCC utilisation, FPO/SHG/Agri-Retail entity types, and geospatial naming.

CLAUDE CODE PROMPT

Phase 1: India Stack & Agri-FinTech Terminology Refactor

Refactor GramOS terminology and data models to align with the Indian Agri-FinTech ecosystem.

1. Update Models (backend/models.py):
 - Rename `alpha_earth_embeddings` to `geospatial_crop_embeddings`.
 - In `FinancialProfile`, add `kcc_limit_utilized_inr` (float, default 0), `kcc_utilization_pct` (float), and set `business_type` to accept 'FPO', 'SHG', or 'Agri-Retail'.
2. Update Mock Data (backend/mock_data.py):
 - Update profiles to use these new fields (FPO, SHG, Agri-Retailer).
3. Update Frontend UI (frontend/app/dashboard/risk-dashboard.tsx & page.tsx):
 - Find and replace UI text: change "AlphaEarth" to "Geospatial / Satellite Data".
 - Update the Input Signals card to display KCC Limit Utilized.

Verify backend runs, Next.js builds cleanly, and charts render correctly.

Phase 2 — RBI-Compliant Deterministic Engine & Parallel LangGraph

Objective. Upgrade the engine to a parallel multi-agent LangGraph system in which the score is purely mathematical and Gemini produces only the explainable narrative.

CLAUDE CODE PROMPT

Phase 2: RBI-Compliant LangGraph & Deterministic Scoring Engine

Upgrade backend/engine.py to a parallel LangGraph multi-agent system. The final risk score MUST be calculated deterministically via a mathematical formula to comply with RBI Model Risk Management guidelines. Gemini will generate ONLY the Explainable AI (XAI) narrative.

1. Install required packages:


```
pip install langgraph langchain-google-genai langchain-core
```

 Update requirements.txt.
2. In backend/engine.py, define `GraphState TypedDict` containing:
 - `request` (`RiskAssessmentRequest`)
 - `climate_analysis` (`str`)
 - `financial_analysis` (`str`)
 - `deterministic_score` (`int`)
 - `final_assessment` (`RiskAssessmentResponse`)
3. Define async node functions:
 - `analyze_climate(state)`: Uses `ChatGoogleGenerativeAI` to evaluate `ClimateProfile` (NDVI, soil moisture, rainfall). Returns string analysis.
 - `analyze_financials(state)`: Uses `ChatGoogleGenerativeAI` to evaluate `FinancialProfile` (UPI, KCC, DPD). Returns string analysis.
 - `calculate_base_score(state)`: Pure deterministic mathematical function.
 Score Formula = $100 - (DPD * 2) - (Rainfall_Deficit * 0.5) - (KCC_Util > 80\% ? 10 : 0)$.
 Clamp between 0 and 100.
 - `synthesize_risk(state)`: Passes analyses and `deterministic_score` to Gemini using


```
.with_structured_output(RiskAssessmentResponse). Instruct Gemini to keep the
deterministic_score FIXED as the final score and generate narrative explanations.
```

4. Build StateGraph with parallel execution:
 - START -> analyze_climate
 - START -> analyze_financials
 - START -> calculate_base_score
 - All three join into synthesize_risk -> END.
5. In synthesize_risk, explicitly overwrite output.risk_score = deterministic_score as a safety backstop against LLM hallucination.
6. Update main.py to invoke this compiled graph for /api/assess-risk.

Phase 3 — Supabase PostgreSQL Integration

Objective. Migrate from local memory / SQLite to Supabase PostgreSQL with an idempotent initialisation script.

CLAUDE CODE PROMPT

Phase 3: Supabase PostgreSQL Database Setup

Migrate GramOS from local memory/SQLite to Supabase PostgreSQL.

1. Install backend dependencies:


```
pip install supabase psycopg2-binary
```

 Update requirements.txt.
2. Create backend/database.py:


```
Initialize Supabase client using SUPABASE_URL and SUPABASE_SERVICE_KEY environment variables.
Expose get_supabase_client().
```
3. Create backend/scripts/init_supabase.py:


```
Execute SQL DDL over psycopg2 connection using DATABASE_URL to create tables:
- enterprises (id UUID PRIMARY KEY, name, business_type, location_district, location_taluk)
- financial_ledgers (id UUID, enterprise_id FK, upi_volume_inr, dpd_days, kcc_utilization_pct,
  recorded_at TIMESTAMP)
- climate_snapshots (id UUID, enterprise_id FK, ndvi_index, soil_moisture_percentage,
  rainfall_deviation_pct, recorded_at TIMESTAMP)
```
4. Make init_supabase.py idempotent (checking existing records before inserting seed data).
 Document credentials in backend/.env.example.

Phase 4 — Time-Series Data Generator (30-Day Historical Backfill)

Objective. Populate thirty days of synthetic history so timeline charts, trend detection and threshold alerting can be tested realistically.

CLAUDE CODE PROMPT

Phase 4: Synthetic Time-Series Backfill Generator

Build a synthetic data generator script to populate 30 days of historical data for testing timeline charts.

1. Create backend/scripts/seed_dynamic_data.py.
2. Alter climate_snapshots and financial_ledgers tables to add a UNIQUE(enterprise_id, recorded_at) constraint for idempotency.
3. Generate 30 days of historical rows per enterprise:
 - For one specific enterprise (e.g., Dairy Cooperative), generate a steady declining trend in NDVI (0.60 -> 0.25) and soil moisture to simulate a drought.
 - For other enterprises, use a mean-reverting random walk anchored around baseline values.
 - Generate corresponding daily financial ledger records (varying transaction counts and DPD trends).
4. Update backend/models.py to add Enterprise, FinancialLedger, and ClimateSnapshot Pydantic models.

Phase 5 — Rewire Backend to Supabase & Dual-Axis Timeline UI

Objective. Point every read path at Supabase and build the dual-axis Recharts timeline spanning historical days and projected months.

CLAUDE CODE PROMPT

Phase 5: Rewire FastAPI to Supabase & Build Dual-Axis Recharts Timeline

1. Backend Updates (backend/main.py):

- Rewrite GET /api/mock-profiles to query the enterprises table in Supabase.
- Rewrite GET /api/mock-profiles/{id} to query the latest financial_ledgers and climate_snapshots from Supabase (ORDER BY recorded_at DESC LIMIT 1).
- Create GET /api/enterprises/{id}/history returning the last 30 daily records ordered ascending.
- Delete backend/mock_data.py.

2. Frontend Updates:

- Update frontend/lib/api.ts with getEnterpriseHistory(id).
- In frontend/components/RiskChart.tsx, build a timeline spanning negative days (historical, -30 to 0) to positive months (projected, +1 to +6).
- Implement dual Y-axes: Left Y-axis for Monthly Revenue (INR), Right Y-axis for NDVI (0.0 - 1.0).
- Render historical revenue as solid Emerald line, historical NDVI as solid Slate line, and projected revenue as dashed line.

04 BUILD SEQUENCE · PART II

Security, Intelligence & Autonomy

Phases 6 through 9 turn a working prototype into a supervised, auditable system: real authentication, multimodal document ingestion, human override with a compliance trail, and an autonomous worker that watches the data while nobody is looking.

Phase 6 — Supabase Auth & JWKS Public Key JWT Verification

Objective. Ensure only authenticated loan officers reach the dashboard and the API, using JWKS public-key verification that supports ECC/P-256 signatures.

CLAUDE CODE PROMPT

Phase 6: Supabase Auth & API Security

Secure GramOS so only authenticated loan officers can access the dashboard and APIs.

1. Frontend Setup:

- Install @supabase/supabase-js and @supabase/ssr in frontend/.
- Create frontend/lib/supabase/client.ts (browser client) and frontend/lib/supabase/server.ts (cookie-backed server client).
- Create frontend/app/login/page.tsx with an email/password form using signInWithPassword.
- Protect frontend/app/dashboard/page.tsx by checking getUser() server-side and calling redirect("/login") if unauthenticated.
- Update frontend/lib/api.ts to attach Authorization: Bearer <access_token> on all API calls.

2. Backend Setup (backend/main.py):

- Create a verify_jwt dependency using PyJWT.
- Fetch public keys dynamically from Supabase JWKS endpoint (https://<project-ref>.supabase.co/rest/v1/rpc/...) to support ECC/P-256 JWT signatures natively.
- Apply verify_jwt dependency to /api/mock-profiles, /api/assess-risk, and /api/enterprises/{id}/history.

Phase 7 — Multimodal Document Intelligence (Gemini 2.5 Flash Vision)

Objective. Let loan officers upload bank statements and land records as PDF or image and get structured JSON extraction instantly.

CLAUDE CODE PROMPT

Phase 7: Multimodal Document Extraction Engine

Build a document intelligence layer allowing loan officers to upload bank statements or land records (PDF/images) for instant AI extraction.

1. Backend Setup:

- Install python-multipart. Update requirements.txt.
- Add document_insights table in Supabase: (id, enterprise_id FK, document_type, extracted_json JSONB, recorded_at).
- In backend/engine.py, create extract_document_insights(file_bytes, mime_type) using the google-genai SDK (genai.Client.aio.models.generate_content) with Part.from_bytes and response_mime_type="application/json".
- In backend/main.py, create POST /api/enterprises/{id}/upload-document (accepting PDF/images up to 20MB) and GET /api/enterprises/{id}/documents.

2. Frontend Setup:

- Create frontend/components/DocumentUploader.tsx with drag-and-drop file support.
- Display extracted JSON key-value metrics in a "Document Insights" card on frontend/app/dashboard/risk-dashboard.tsx.

Phase 8 — Human-in-the-Loop Audit Log & Compliance Overrides

Objective. Satisfy central-bank expectations of human oversight: every override is bound to an authenticated officer identity and a written justification.

CLAUDE CODE PROMPT

Phase 8: Human-in-the-Loop Audit Log & Manual Overrides

Build an audit log and manual override workflow to satisfy central bank requirements for human oversight.

1. Backend Setup:

- Add audit_logs table in Supabase: (id, enterprise_id FK, officer_id, original_score, overridden_score, justification, recorded_at).
- In backend/main.py, create POST /api/enterprises/{id}/override-score. Extract officer_id directly from the validated JWT sub claim. Validate justification is non-empty.

2. Frontend Setup:

- Create frontend/components/OverrideScoreModal.tsx requiring a new score (0-100) and text justification.
- Add a "Manual Override" button to the risk score gauge on the dashboard.
- Update the score gauge to display the overridden score with a "Human Overridden" badge and toast confirmation.

Phase 9 — Proactive Background Risk Alerting

Objective. Run an autonomous worker that scans incoming data and raises threshold-breach alerts without waiting for an officer to open the dashboard.

CLAUDE CODE PROMPT

Phase 9: Autonomous Background Worker for Proactive Risk Alerts

Build a background worker that monitors data streams and flags risk threshold breaches autonomously.

1. Backend Setup:

- Install apscheduler. Update requirements.txt.
- Add alerts table in Supabase: (id, enterprise_id FK, alert_type, message, is_read, created_at).
- Create backend/worker.py with check_climate_thresholds(). Scan the latest climate_snapshots for every enterprise; if ndvi_index < 0.30 or soil_moisture_percentage < 20.0%, insert a CLIMATE_WARNING row (deduplicated to 1 per day).
- In backend/main.py, initialize AsyncIOScheduler in the lifespan context manager, running check_climate_thresholds every 1 minute.
- Create GET /api/enterprises/{id}/alerts.

2. Frontend Setup:

- Display a red "Climate Warning" banner at the top of the dashboard whenever unread alerts exist.

05 THE REAL-DATA PIVOT

Sugarcane Arrears & Buyer Risk Engine

These phases replace synthetic climate metrics with real sugarcane payment arrears data sourced from the Karnataka Cane Commissionerate. This is the pivot that turns GramOS from a well-built demo into a platform with a defensible, proprietary signal.

Phase 10 — Cane Arrears & Catchment Database Integration

Objective. Bring real mill arrears and catchment mapping into Supabase as first-class tables, with a seeded set of Belagavi, Bagalkot and Vijayapura figures.

Seed Data — Karnataka 2025–26 Cane Commissionerate Figures

Mill	District	Payable	Paid	Outstanding	Arrears %
Mill A	Belagavi	₹250 Cr	₹140 Cr	₹110 Cr	44.0%
Mill B	Bagalkot	₹180 Cr	₹90 Cr	₹90 Cr	50.0%
Mill C	Vijayapura	₹120 Cr	₹110 Cr	₹10 Cr	8.3%

CLAUDE CODE PROMPT

Phase 10: Sugarcane Arrears & Catchment Database Schema

Integrate real sugarcane mill arrears and catchment mapping into Supabase.

- Database Migration (backend/scripts/init_supabase.py):
Add tables:
 - sugar_mills (
 - id UUID PRIMARY KEY,
 - mill_name VARCHAR NOT NULL,
 - district VARCHAR NOT NULL,
 - taluk VARCHAR NOT NULL,
 - season VARCHAR DEFAULT '2025-26',
 - dues_payable_inr_cr NUMERIC(10,2),
 - dues_paid_inr_cr NUMERIC(10,2),
 - outstanding_arrears_inr_cr NUMERIC(10,2),
 - arrears_pct NUMERIC(5,2),
 - source_tier VARCHAR,
 - as_of_date DATE
 - mill_catchments (
 - id UUID PRIMARY KEY,
 - mill_id UUID REFERENCES sugar_mills(id) ON DELETE CASCADE,
 - catchment_taluk VARCHAR NOT NULL,
 - supply_share_pct NUMERIC(5,2) DEFAULT 100.0
- Create Seed Script (backend/scripts/seed_cane_arrears.py):
Seed official Karnataka 2025-26 Cane Commissionerate figures for Belagavi, Bagalkot, and Vijayapura:
 - Example Mills:
 - * Mill A (Belagavi): Payable Rs 250 Cr, Paid Rs 140 Cr, Outstanding Rs 110 Cr (Arrears 44.0%).
 - * Mill B (Bagalkot): Payable Rs 180 Cr, Paid Rs 90 Cr, Outstanding Rs 90 Cr (Arrears 50.0%).
 - * Mill C (Vijayapura): Payable Rs 120 Cr, Paid Rs 110 Cr, Outstanding Rs 10 Cr (Arrears 8.3%).
 - Map catchment_taluk linkages for each mill.
- Update Models (backend/models.py):
Add SugarMill, MillCatchment, and BuyerPaymentRiskProfile Pydantic models.

Run the seed script and verify database population.

Phase 11 — Integrating buyer_payment_risk into Scoring & LangGraph

Objective. Wire buyer payment risk into both halves of the system: the deterministic formula and the narrative agents.

CLAUDE CODE PROMPT

Phase 11: Integrate Buyer Payment Risk into Scoring Engine & LangGraph

Incorporate buyer payment risk into the deterministic mathematical model and AI narrative agents.

1. Buyer Risk Calculation (backend/engine.py):
 - Create `calculate_buyer_payment_risk(enterprise_taluk: str) -> dict:`
 - Query `mill_catchments` for mills drawing cane from `enterprise_taluk`.
 - Compute Weighted Arrears Exposure:

$$\text{weighted_exposure} = \text{SUM}(\text{supply_share_pct} \times \text{outstanding_arrears_inr_cr})$$
 - Compute Buyer Risk Score Penalty (0 to 40 points):

$$\text{buyer_penalty} = \min(40, (\text{weighted_exposure} / 100) \times 25)$$
2. Update Deterministic Risk Score Formula:

$$\text{Final Risk Score} = \max(0, 100 - (\text{DPD} \times 2) - (\text{KCC_Util} > 80\% ? 10 : 0) - \text{buyer_penalty})$$
3. Update LangGraph State & Analyst Node:
 - Add `buyer_risk_analysis` to `GraphState`.
 - Replace or augment `analyze_climate` with `analyze_buyer_risk(state):` Evaluate mill arrears position, payment lag (60-120 days), and cash-flow shock impact.
 - Update `synthesize_risk` node to incorporate buyer payment stress into the final XAI mitigation summary.
4. Update FastAPI Endpoint:
 - Ensure `POST /api/assess-risk` fetches the enterprise's taluk, calculates `buyer_payment_risk`, and passes it into the LangGraph state machine.

Phase 12 — Buyer Payment Risk UI & Live Production Deployment

Objective. Surface the signal on the dashboard and ship the full stack to production on Vercel and Railway / Render.

CLAUDE CODE PROMPT

Phase 12: Buyer Payment Risk UI Dashboard & Production Deployment

Expose the real mill arrears risk signal on the Next.js dashboard and deploy the full-stack platform to production.

1. Frontend UI Updates (frontend/app/dashboard/risk-dashboard.tsx):
 - Add a "Buyer Payment Risk (Sugar Mill Arrears)" card to the Input Signals section.
 - Display: Primary Mill Name, Catchment Outstanding Arrears (Rs Cr), Mill Payment Lag Status, and Calculated Buyer Risk Factor.
 - Add a warning badge if catchment arrears exceed Rs 50 Cr ("High Supply-Chain Cashflow Stress").
2. Deployment Setup:
 - Frontend (Vercel):
 - * Push repository to GitHub.
 - * Import frontend/ into Vercel.
 - * Set Environment Variables: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_API_BASE_URL` (Railway URL).
 - Backend (Railway / Render):
 - * Create new web service connected to backend/.
 - * Configure Build Command: `pip install -r requirements.txt`.
 - * Configure Start Command: `uvicorn main:app --host 0.0.0.0 --port $PORT`.
 - * Set Environment Variables: `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `DATABASE_URL`, `GEMINI_API_KEY`.
3. End-to-End Verification:
 - Verify production deployment URL loads cleanly.

- Confirm login flow, buyer payment risk extraction, LangGraph execution, and PDF document uploads operate smoothly in production.

06 REFERENCE

Scoring Methodology

The score is deterministic by design. Every input below is a number computed from stored data; no part of the final figure is authored by a language model. The LLM layer produces the explanation that accompanies the number, and the synthesiser node overwrites the model's score field with the deterministic value before the response is returned.

6.1 Weighted Arrears Exposure

For a borrower in a given taluk, query `mill_catchments` for every mill drawing cane from that taluk, then sum each mill's outstanding arrears weighted by its share of supply:

```
weighted_exposure = SUM( supply_share_pct × outstanding_arrears_inr_cr )
```

6.2 Buyer Risk Penalty

Exposure converts into a capped penalty of 0 to 40 points:

```
buyer_penalty = min( 40 , ( weighted_exposure / 100 ) × 25 )
```

6.3 Final Deterministic Risk Score

```
Final Risk Score = max( 0 , 100
    - ( DPD × 2 )
    - ( KCC_Util > 80% ? 10 : 0 )
    - buyer_penalty )
```

The earlier Phase 2 formulation, retained for climate-only assessments, was:

```
Score = 100 - ( DPD × 2 ) - ( Rainfall_Deficit × 0.5 ) - ( KCC_Util > 80% ? 10 : 0 )
    clamped between 0 and 100
```

6.4 Component Summary

Component	Driver	Weight / Cap
DPD impact	Days past due on existing obligations	2 points per day
KCC utilisation	Kisan Credit Card limit utilised above 80%	Flat 10 points
Buyer payment risk	Supply-weighted mill arrears in the catchment	Capped at 40 points
Climate stress	Rainfall deficit / NDVI deterioration	0.5 points per unit deficit

Alert thresholds (Phase 9 worker). A `CLIMATE_WARNING` row is raised when `ndvi_index` < 0.30 or `soil_moisture_percentage` < 20.0%, deduplicated to one alert per enterprise per day. On the dashboard, catchment arrears above ₹50 Cr raise a *High Supply-Chain Cashflow Stress* badge.

07 GO-LIVE

Production Readiness Checklist

Every item below is implemented in the current build. Treat this as the pre-demo and pre-deployment verification sheet.

✓	RBI compliance	Deterministic scoring logic fully separated from LLM generation.
✓	Security	Auth via Supabase JWKS public keys; all routes protected by JWT verification.
✓	Data provenance	Sourced from Karnataka Cane Commissionerate public records.
✓	Resilience	Async timeout wrappers with golden fallback caching.
✓	Multimodal capability	Native Gemini 2.5 Flash document extraction for PDFs and image scans.
✓	Auditability	Human-in-the-loop override log bound to authenticated user IDs.
✓	Autonomous alerting	APScheduler background worker continuously monitoring risk thresholds.

A APPENDIX

Schema & Endpoint Index

A consolidated index of everything the build sequence creates, for quick lookup during development, debugging or a technical due-diligence conversation.

A.1 Database Tables

Table	Introduced	Key columns
enterprises	Phase 3	id UUID PK, name, business_type, location_district, location_taluk
financial_ledgers	Phase 3	id UUID, enterprise_id FK, upi_volume_inr, dpd_days, kcc_utilization_pct, recorded_at
climate_snapshots	Phase 3	id UUID, enterprise_id FK, ndvi_index, soil_moisture_percentage, rainfall_deviation_pct, recorded_at
document_insights	Phase 7	id, enterprise_id FK, document_type, extracted_json JSONB, recorded_at
audit_logs	Phase 8	id, enterprise_id FK, officer_id, original_score, overridden_score, justification, recorded_at
alerts	Phase 9	id, enterprise_id FK, alert_type, message, is_read, created_at
sugar_mills	Phase 10	id UUID PK, mill_name, district, taluk, season, dues_payable_inr_cr, dues_paid_inr_cr, outstanding_arrears_inr_cr, arrears_pct, source_tier, as_of_date
mill_catchments	Phase 10	id UUID PK, mill_id FK (ON DELETE CASCADE), catchment_taluk, supply_share_pct

A.2 API Endpoints

Method & path	Purpose
GET /api/mock-profiles	List enterprises from Supabase
GET /api/mock-profiles/{id}	Latest financial ledger and climate snapshot for one enterprise
GET /api/enterprises/{id}/history	Last 30 daily records, ascending
POST /api/assess-risk	Run the compiled LangGraph assessment, including buyer payment risk
POST /api/enterprises/{id}/upload-document	Multimodal PDF / image extraction, up to 20MB
GET /api/enterprises/{id}/documents	Retrieve stored document insights
POST /api/enterprises/{id}/override-score	Human-in-the-loop override, officer_id from JWT sub
GET /api/enterprises/{id}/alerts	Unread and historical risk alerts

A.3 Environment Variables

Service	Variables
Backend (Railway / Render)	SUPABASE_URL, SUPABASE_SERVICE_KEY, DATABASE_URL, GEMINI_API_KEY
Frontend (Vercel)	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, NEXT_PUBLIC_API_BASE_URL



GramOS · Master Blueprint & Engineering Manual · Version 2.0 · Maintained by Sudarshan Kulkarni · Karnataka